Advanced Rust 2026 - Lab 6: Systems Interfaces - Teacher Copy

Lukáš Hozda

Spring 2026

Exercise 1: C String Boundary

Questions and Answers

1. Why must `strlen_checked` check for a NUL byte before calling `libc::strlen`?

`strlen` walks memory until it finds a zero byte. If the Rust slice is not terminated, C will keep reading past the slice boundary. Rust's slice length is not visible to C, so the safe wrapper must check the C precondition first.

1. Why is `CString::new` the right construction path from `&str`?

A C string cannot contain an interior NUL because C APIs interpret the first NUL as the end of the string. `CString::new` checks this and reports the byte index of the NUL.

1. Why is invalid UTF-8 a separate error when parsing a block back to Rust?

`CStr` only describes NUL-terminated bytes. It does not imply UTF-8. Converting to `String` adds the Rust string invariant that the bytes must be valid UTF-8.

Exercise 2: Anonymous Memory Mapping

Questions and Answers

1. Why is `MAP_FAILED` checked instead of null?

The POSIX `mmap` failure sentinel is `(void*) -1`, represented by `libc::MAP_FAILED`. A null pointer is not the documented failure value.

1. Why reject zero-length mappings?

A zero-length mapping is not useful for the wrapper and complicates the invariant that `ptr` and `len` describe a live mapping. Rejecting it makes the rest of the type simpler: every successful value has `len > 0`.

1. Why must `munmap` receive the same pointer and length?

The mapping is an OS resource, not a Rust allocation. The kernel tracks the virtual memory region by base address and length. Passing a derived pointer or a different length breaks the resource ownership contract.

1. What must be true when returning `&mut [u8]` from the mapping?

The mapping must still be live, the pointer and length must describe valid mapped memory, and no other active references may alias the same bytes in a way that violates Rust's aliasing rules.

Exercise 3: Pinned Self-Referential Record

Questions and Answers

1. Why is the naive constructor unsound?

If a struct stores a pointer to one of its own fields before it reaches its final address, moving the struct invalidates the pointer. Returning a local value from `new` may move it.

1. What does `Pin<Box<T>>` guarantee here?

The `Box` allocation gives the value a stable heap address. `Pin` plus `PhantomPinned` prevents safe code from later moving the `T` out of that pinned location.

1. Why is `get_unchecked_mut` needed during construction and rename?

The compiler cannot prove that mutating these fields preserves the pinning invariant. The implementation has to assert that it does not move the pinned value itself and keeps `name_ptr` consistent.

1. Why is moving the `Pin<Box<StableRecord>>` variable still fine?

Moving the pointer object does not move the heap allocation it points to. The pointee remains at the same address.

Cargo.toml

```
[package]
name = "advanced-lab-06-systems-interfaces"
version = "0.1.0"
edition = "2021"

[dependencies]
libc = "0.2"
```

Full Reference Implementation

```
pub mod c_strings {
    use std::ffi::{CStr, CString};

    #[derive(Debug, Clone, PartialEq, Eq)]
    pub enum CStringError {
        InteriorNul(usize),
        MissingTerminator,
        InvalidUtf8,
    }

    pub fn to_c_bytes(input: &str) -> Result<Vec<u8>, CStringError> {
        match CString::new(input) {
            Ok(c) => Ok(c.into_bytes_with_nul()),
            Err(err) => Err(CStringError::InteriorNul(err.nul_position())),
        }
    }

    pub fn strlen_checked(bytes: &[u8]) -> Result<usize, CStringError> {
        if !bytes.contains(&0) {
            return Err(CStringError::MissingTerminator);
        }
        Ok(unsafe { libc::strlen(bytes.as_ptr().cast()) as usize })
    }

    pub fn split_c_string_block(bytes: &[u8]) -> Result<Vec<String>, CStringError> {
        if bytes.is_empty() {
            return Ok(Vec::new());
        }
        if bytes.last() != Some(&0) {
            return Err(CStringError::MissingTerminator);
        }

        let mut out = Vec::new();
        let mut start = 0;
        for (idx, byte) in bytes.iter().copied().enumerate() {
            if byte == 0 {
                let c = CStr::from_bytes_with_nul(&bytes[start..=idx])
                    .map_err(|_| CStringError::InteriorNul(start))?;
                let text = c.to_str().map_err(|_| CStringError::InvalidUtf8)?;
                out.push(text.to_string());
                start = idx + 1;
            }
        }
        Ok(out)
    }

    #[cfg(test)]
    mod tests {
        use super::*;
```

```
        #[test]
        fn builds_c_bytes() {
            assert_eq!(to_c_bytes("abc").unwrap(), b"abc\0");
            assert_eq!(to_c_bytes("a\0b"), Err(CStringError::InteriorNul(1)));
        }

        #[test]
        fn checks_strlen_preconditions() {
            assert_eq!(strlen_checked(b"abc\0ignored").unwrap(), 3);
            assert_eq!(strlen_checked(b"abc"), Err(CStringError::MissingTerminator));
        }

        #[test]
        fn splits_c_string_blocks() {
            assert_eq!(
                split_c_string_block(b"red\0\0blue\0").unwrap(),
                vec!["red".to_string(), "".to_string(), "blue".to_string()]
            );
            assert_eq!(
                split_c_string_block(b"red\0blue"),
                Err(CStringError::MissingTerminator)
            );
            assert_eq!(
                split_c_string_block(&[0xFF, 0]),
                Err(CStringError::InvalidUtf8)
            );
        }
    }
}

pub mod mapping {
    use std::io;
    use std::ptr::NonNull;

    pub struct Mapping {
        ptr: NonNull<u8>,
        len: usize,
    }

    impl Mapping {
        pub fn anonymous(len: usize) -> io::Result<Self> {
            if len == 0 {
                return Err(io::Error::new(io::ErrorKind::InvalidInput, "zero-length mapping"));
            }

            let raw = unsafe {
                libc::mmap(
                    std::ptr::null_mut(),
                    len,
                    libc::PROT_READ | libc::PROT_WRITE,
                    libc::MAP_PRIVATE | libc::MAP_ANONYMOUS,
                    -1,
                    0,
                )
            };
            if raw == libc::MAP_FAILED {
                return Err(io::Error::last_os_error());
            }

            let Some(ptr) = NonNull::new(raw.cast()) else {
                unsafe {
                    libc::munmap(raw, len);
                }
                return Err(io::Error::other("mmap returned null"));
            };
```

```
            Ok(Self { ptr, len })
        }

        pub fn len(&self) -> usize {
            self.len
        }

        pub fn as_slice(&self) -> &[u8] {
            unsafe { std::slice::from_raw_parts(self.ptr.as_ptr(), self.len) }
        }

        pub fn as_mut_slice(&mut self) -> &mut [u8] {
            unsafe { std::slice::from_raw_parts_mut(self.ptr.as_ptr(), self.len) }
        }

        pub fn fill(&mut self, value: u8) {
            self.as_mut_slice().fill(value);
        }
    }

    impl Drop for Mapping {
        fn drop(&mut self) {
            unsafe {
                libc::munmap(self.ptr.as_ptr().cast(), self.len);
            }
        }
    }

    #[cfg(test)]
    mod tests {
        use super::*;
        use std::io::ErrorKind;

        #[test]
        fn maps_and_mutates_memory() {
            let mut mapping = Mapping::anonymous(16).unwrap();
            assert_eq!(mapping.len(), 16);
            mapping.fill(0xAB);
            assert_eq!(mapping.as_slice(), &[0xAB; 16]);
            mapping.as_mut_slice()[3] = 7;
            assert_eq!(mapping.as_slice()[3], 7);
        }

        #[test]
        fn rejects_zero_length() {
            let err = match Mapping::anonymous(0) {
                Ok(_) => panic!("zero-length mapping succeeded"),
                Err(err) => err,
            };
            assert_eq!(err.kind(), ErrorKind::InvalidInput);
        }
    }
}

pub mod pinned_record {
    use std::marker::PhantomPinned;
    use std::pin::Pin;
    use std::ptr;

    pub struct StableRecord {
        name: String,
        name_ptr: *const String,
        _pin: PhantomPinned,
    }
```

```
    impl StableRecord {
        pub fn new(name: String) -> Pin<Box<Self>> {
            let mut boxed = Box::pin(Self {
                name,
                name_ptr: ptr::null(),
                _pin: PhantomPinned,
            });

            let name_ptr = &boxed.name as *const String;
            unsafe {
                Pin::as_mut(&mut boxed).get_unchecked_mut().name_ptr = name_ptr;
            }
            boxed
        }

        pub fn name(self: Pin<&Self>) -> &str {
            &self.get_ref().name
        }

        pub fn pointer_matches(self: Pin<&Self>) -> bool {
            ptr::eq(&self.get_ref().name, self.get_ref().name_ptr)
        }

        pub fn rename(self: Pin<&mut Self>, name: String) {
            let this = unsafe { self.get_unchecked_mut() };
            this.name = name;
            this.name_ptr = &this.name as *const String;
        }
    }

    #[cfg(test)]
    mod tests {
        use super::*;

        #[test]
        fn keeps_internal_pointer_valid() {
            let mut record = StableRecord::new("alpha".to_string());
            assert_eq!(record.as_ref().name(), "alpha");
            assert!(record.as_ref().pointer_matches());
            record.as_mut().rename("beta".to_string());
            assert_eq!(record.as_ref().name(), "beta");
            assert!(record.as_ref().pointer_matches());

            let moved_pin = record;
            assert_eq!(moved_pin.as_ref().name(), "beta");
            assert!(moved_pin.as_ref().pointer_matches());
        }
    }
}
```